Week 4 – Display, serial output & PWM

This lab is the last of three labs that help you complete your first project milestone (MS1) due in week 5. **At this point, you should have completed all the exercises in the week 3 lab and committed the associated code to version control.**

1. Lab objectives

- Use the OLED display on the RCAP
- Direct standard output to UART for debugging
- Configure STM32 timers to output PWM signals

2. Configure the OLED Display

We will use an existing library to operate the SSD1306 OLED display on your RCAP board. This library uses I²C, so we will need to configure this first. Because the MCU needs to periodically communicate a large amount of data to refresh the display, we will use the DMA (as we have done with the ADC) to reduce the load on the CPU.[1]

1. Use the schematics provided to you to fill in the following table:

User label	**GPIO pin ID**	**MCU I2C Module** (I2C1/I2C2)
OLED_I2C_SDA		
OLED_I2C_SCL		

2. Open your project configuration on STM32CubeMX.
3. In the pinout view, select the GPIO pins you have identified, and configure them with the corresponding I2C function. The pins will be shaded yellow.
4. On the left-hand pane, expand "Connectivity" and select the I2C module you have identified.
5. Under "Mode", select "I2C" instead of "Disable". The pins will now turn green.
6. Under configuration, open the NVIC Settings tab and enable the I2C module's interrupt.
7. Under configuration, open DMA Settings and click "Add".
8. Under DMA request, select I2Cx_TX.

You should be able see that DMA1 Channel 2 is set up to transfer data from memory to peripheral (which is I2C).

9. Leave all other settings as default and generate the code.
10. From your lab files, identify the files starting with "ssd1306",[2] copy the header files into the *Inc* directory in your project, and the C files into the *Src* directory.
11. Follow the same pattern as you have done in the previous lab to create a task module for display.
12. In the initialisation function of the module, add the following code:

```
ssd1306_Init();
ssd1306_SetCursor(0, 0);
ssd1306_WriteString("Hello world!", Font_7x10, White);
```

[1] Bench testing has shown that using the DMA reduces the time it takes the CPU to refresh the display to 1.5 ms down from a whopping 100 ms. Consider the impact this would have had on the CPU's ability to execute the other tasks on time.

[2] This is the OLED screen part number.

13. In the execute function of the module, add the following code:

```
ssd1306_UpdateScreen();
```

14. Plumb the new task into your task scheduler and make it run at 4 Hz.

Build, run and test your code. You should be able to see "Hello world!" displayed at the top of your screen. If all's well, you know what to do![3]

3. Live updates on screen

The next step is to update the screen with real time information! Use the following functions to update the display with the current joystick ADC values, each on a separate line, when the display task is executed:

- `ssd1306_SetCursor()` to change the cursor location on the screen.
- `ssd1306_WriteString()` to print a string to the screen starting from the current cursor location.
- **`snprintf()`**[4] to format a string with runtime variables before passing it to `ssd1306_WriteString()`.

Hint: to communicate between modules, use getter functions, which simply return a variable.

4. Serial debugging

In embedded systems, it is common practice to periodically print device state information to serial output for troubleshooting and analysis. Luckily, your Nucleo board has an integrated serial link that uses the same USB port as the programmer.

Refer to the Nucleo-C071RB user manual (MB2046) and find the section on Virtual COM port (VCP) to fill in the following table:

User label	**GPIO pin ID**	**MCU USART Module** (USART1/USART2)
VCP_USART_TX		
VCP_USART_RX		

Go through the steps, similar to section 2, to set up the pins you have identified with the corresponding USART function and enable the correct USART unit in *Asynchronous* mode. You can assume that all other settings can be left as default.

Once you have generated the new code, print the ADC values to serial UART in the display task using the following HAL function:

```
HAL_UART_Transmit(…)
```

Use Ctrl+H in the IDE to find where this function is defined and how to use it. You may also want to refer to the HAL reference manual on learn.

Once you have implemented your code and loaded it on your device, use Tera Term (or any terminal software) to test it.

1. Open Tera Term and select "Serial".

[3] You know to commit and push when your code works, surely? Since you are now something of a git expert (until your next lecture on git), you will no longer be prompted for this.
[4] This is a C standard library function, so you can look it up online to find example usages.

2. Choose the port labelled "STMicroelectronics STLink Virtual COM port"

By default, you should see what seems to be a rubbish stream of output. To fix this:

3. Open *Setup -> Serial port*...
4. In the *Speed* dropdown menu select 115200, which matches the default baud rate you have configured your project with.
5. If line separation doesn't seem to work correctly, check that you have both a carriage return (CR) and a line feed (LF) characters at the end of your printed strings i.e. "`\r\n`".

The code can be further simplified by directing standard output to the UART peripheral, allowing you to simply use *printf* instead of making an explicit function call to the UART HAL library. You can optionally do this by following the instructions in *Appendix A*.

5. Adjusting the brightness with PWM

Pulse-width modulation (PWM) is a powerful technique for varying the width of a rectangular wave in a digital signal. One useful application of this is to vary the average power delivered to an LED to adjust its brightness. In this exercise, you will vary the brightness on your DS3 using PWM. For a microcontroller to produce a rectangular wave with a specific frequency and duty cycle, it uses hardware timers to keep track of time. The pin of interest is internally connected to the hardware timer module #2 through channel 3 on the microcontroller.

1. In your STM32CubeMX configurator, change the pin that corresponds to DS3 to use TIM2_CH3, instead of being configured as a GPIO_Output.
2. Open the TIM2 settings (under Timers),
3. Next to *Channel3*, select *PWM Generation CH3*
4. In the *Parameter Settings* at the bottom of the window, choose a *Counter Period* of 100,000 and a *Pulse (32 bits value)* of 25,000.
5. Make sure *CH Polarity* is set to "Low" since the hardware configuration of the LED is active-low.

Based on these settings what do you think the duty cycle of the LED would be initialised to?

Once you have generated the new code, use the function *HAL_TIM_PWM_Start* to start the PWM on timer module 2 channel 3. Inspect the HAL library code and the STM32C0 HAL manual on Learn to determine the correct arguments to pass into the function. You should be able to see the DS3 glowing faintly, corresponding to the duty cycle you have determined up.

Note: since DS3 is no longer configured as a GPIO, trying to use the *rgb* library for operating DS3 may interfere with the PWM setup. You will need to find a way to work around this or adapt the *rgb* library to support PWM.

Finally, modify your code so that each SW1 press increases the duty cycle of DS3 by increments of 25% and wraps back to 0% when it exceeds 100%. You may need to use the files **pwm.c** and **pwm.h** provided to you on learn to assist you with this. The functionality of the other buttons should remain unchanged (simply toggling the corresponding LEDs).

6. Getting ready for MS1

By now you should have met most requirements for milestone 1. This is a good time to carefully test your code against these requirements and make any necessary improvements.

Appendix A: UART as the new standard output

Have you ever wondered how the standard library function *printf* knows where to direct strings passed to it? All the system needs to know is what to do with a single byte going into the output stream. In STM32, we can configure this by writing a custom definition of the function **`__io_putchar()`**.[5]

To tie the standard output in your program to the UART peripheral for serial debugging, **in a new C file**, simply define the following function:

```
int __io_putchar(int ch)
{
        HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
        return ch;
}
```

You can now replace the calls to `HAL_UART_Transmit` in the rest of your code with calls to `printf`.

While this neat programming trick simplifies your code, it does come with a catch: the HAL will send only one character at a time, increasing the amount of overhead required to send a full string. This is a trade-off you should assess for yourself.

[5] Similarly, you can tell the program where to get standard input from by defining the function **__io_getchar()**, e.g. so you can customise the behaviour of the standard library function *scanf*.